

APOSTILA



PROGRAMAÇÃO ORIENTADA A OBJETOS

EM **JAVA**

Conceitos, Princípios e Prática
para Desenvolver Soluções Modernas



Marcos Monteiro Junior
PROFESSOR

Programação Orientada a Objetos em Java

Marcos Monteiro Junior

Conceitos Princípios e Prática

1 Organização do projeto, tratamento de exceções e conexão com banco de dados

A medida que um projeto vai crescendo, a organização do código se torna cada vez mais importante. Separar classes que possuem propósitos parecidos, com as demais classes, facilita a manutenção do sistema, além de tornar o código mais legível e compreensível.

1.1 Pacotes

A linguagem Java oferece recursos que permitem organizar o código de forma eficiente, facilitando a manutenção e a compreensão do sistema. O nome desse recurso é **pacotes** (*packages*).

“Um pacote é uma organização física ou lógica criada para separar classes com responsabilidades distintas. Com isso, espera-se que a aplicação fique mais organizada e seja possível separar classes de finalidades e representatividades diferentes” (Leite et al. (2016)).

Cada linguagem possui uma forma de nomear e organizar os pacotes, mas o conceito é o mesmo. Em Java, os pacotes são representados por diretórios, e cada classe é um arquivo dentro do diretório correspondente ao pacote. A convenção de nomenclatura de pacotes em Java é utilizar letras minúsculas e separar palavras com pontos (.), como por exemplo: `com.exemplo.meupacote`.

Para que as classes de pacotes diferentes possam se comunicar, é necessário importar as classes desejadas. A palavra-chave `import` é utilizada para importar classes de outros pacotes, permitindo que elas sejam utilizadas no código.

Observação

O encapsulamento `protect` faz com que todas classes do pacote tenham acesso aos atributos e métodos independente se as classes utilizam a herança ou não.

1.2 Padrão arquitetural MVC (*Model, View, Controller*)

O MVC não é uma tecnologia, linguagem ou biblioteca. É um Padrão de Arquitetura de Software (Design Pattern) criado na década de 1970 (Smalltalk) que revolucionou a forma como construímos sistemas. O seu princípio fundamental é a Separação de Responsabilidades.

1.2.1 A camada model

É o núcleo da aplicação. Representa a “verdade” do sistema. Nessa camada encontram-se as classes de negócio, as regras, a lógica financeira, o estado (atributos) e a comunicação com o Banco de Dados. nessa camada não pode conter nenhuma linha de código visual. O Model não sabe se o sistema roda em Windows, Android ou Web. Ela é cega para o mundo exterior.

1.2.2 A camada controller

É a camada que contém toda a lógica e a ponte de ligação. A maior responsabilidade do controller é escutar os eventos da View (ex: “O usuário clicou em um botão”). Pega essa informação e envia para o Model (ex: “Model, tente realizar a operação”). O Model processa, retorna um resultado (Sucesso ou Falha), e o Controller diz à View: “View, exiba a mensagem de Sucesso e atualize o valor na tela”.

1.2.3 A camada view

É a interface gráfica, a comunicação com o usuário. Nela ficam os códigos de renderização de telas (tabelas, botões, campos de texto, HTML, CSS). Em hipótese alguma essa camada deve conter as regras de negócio. A View nunca deve calcular um saldo ou decidir se um saque é válido, em sistema de gerenciamento de bancário. Ela é “burra” por design. O único trabalho dessa camada é exibir o que o Controller manda e avisar o Controller quando o usuário clica ou digita algo.

1.2.4 Por que usar MVC?

Apesar de ser um design de programação antigo, ele é muito utilizado nos frameworks atuais como SpringBoot, Laravel, ASP.NET, Django e Ruby. Além disso, o MVC proporciona:

- Desenvolvimento Paralelo: Enquanto um desenvolvedor cria as telas (View - Frontend), outro pode programar as regras matemáticas (Model - Backend) simultaneamente, pois as camadas não dependem do código pronto uma da outra.
- Reutilização de Código: O mesmo Model (ContaCorrente) pode ser usado por uma View em Java Swing, um aplicativo Android e um site Web. A regra do banco é a mesma, só a tela muda.
- Manutenibilidade: Se houver um erro de cálculo, o programador sabe que deve procurar no Model. Se um botão estiver desalinhado, ele procura na View. Os “bugs” ficam isolados.
- Facilidade de Testes: É possível criar Testes Automatizados para o Model garantindo que a matemática do banco funciona, sem precisar abrir janelas ou clicar em botões durante o teste.

Observação

Existem diversas arquiteturas de desenvolvimento de software. A escolha do MVC se dá pela sua organização e fácil visualização das responsabilidades de todas as camadas de software.

1.3 Tratamento de exceções

O tratamento de exceções em Java é um mecanismo robusto para lidar com situações inesperadas que ocorrem durante a execução de um programa (como erros de entrada de dados, falhas de conexão ou problemas de leitura de arquivos), permitindo que o sistema não trave abruptamente e possa tomar ações corretivas.

Um programador pode capturar as exceções predefinidas que Java oferece, ou pode criar suas próprias exceções.

Um exemplo de exceção predefinida por Java, em valores lidos pela classe `Scanner`.

```
package br.senac.marcos;

import java.util.Scanner;

public class Exemplo {
    public static void main(String[] args){
        Scanner le = new Scanner(System.in);
        int a;

        System.out.println("Digite um número: ");
        a = le.nextInt();
        System.out.println("Li o valor "+a);
    }
}
```

Se um usuário digitar uma valor de tipo diferente, a aplicação será terminada e retornará a seguinte mensagem

Mensagem

```
Digite um número:
2.5
Exception in thread "main" java.util.
InputMismatchException
at java.util.Scanner.throwFor(Scanner.java:864)
at java.util.Scanner.next(Scanner.java:1485)
at java.util.Scanner.nextInt(Scanner.java:2117)
at java.util.Scanner.nextInt(Scanner.java:2076)
at br.uepg.deinfo.msouza.Exemplo.main(Exemplo.java:13)
```

Essa mensagem é retornada pela máquina virtual, a exceção é a `InputMismatchException`

1 Organização do projeto, tratamento de exceções e conexão com banco de dados

do pacote `java.util`. Para evitar que a aplicação finalize, o programador pode capturar a exceção com os comandos `try-catch-finally`.

A Estrutura `try-catch-finally` é base do tratamento de exceções em Java:

- `try`: Define o bloco de código onde uma exceção pode ocorrer.
- `catch`: Captura a exceção específica que ocorreu no `try` e permite executar um código de tratamento (ex: exibir uma mensagem de erro ao usuário).
- `finally`: É um bloco opcional que será executado sempre, independentemente de uma exceção ter sido lançada ou não. É ideal para fechar conexões com bancos de dados ou liberar recursos de memória.

Um exemplo de exceção é apresentado abaixo:

```
public class ExemploExcecao {
    public static void main(String[] args) {
        try {
            int resultado = 10/0; // Isso causará uma
                                ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Erro: Não é possível dividir por
                                zero!");
        } finally {
            System.out.println("Execucao finalizada.");
        }
    }
}
```

Conceitos

- **Checked Exceptions (Verificadas)**: São exceções que o compilador obriga você a tratar (como `IOException`). Você deve usar `try-catch` ou declarar que o método lança a exceção usando `throws`.
- **Unchecked Exceptions (Não verificadas)**: São erros de lógica ou de tempo de execução (como `NullPointerException` ou `ArrayIndexOutOfBoundsException`). O compilador não exige tratamento, mas o ideal é evitá-las através de boas práticas de programação.
- `throw`: Utilizado dentro do método para disparar manualmente uma exceção.
- `throws`: Utilizado na assinatura do método para avisar que aquele método pode lançar uma exceção, transferindo a responsabilidade do tratamento para quem o chamou.

A correção do código exemplo usando tratamento de exceções pode ser feita da seguinte maneira:

```
import java.util.Scanner;
import java.util.InputMismatchException;
public class Exemplo {
    public static void main(String[] args){
        Scanner le = new Scanner(System.in);
        int a;
        do {
            try{
                System.out.println("Digite um número:");
                a = le.nextInt();
                System.out.println("Li o valor "+a);
                return;
            }
            catch(InputMismatchException ex){
                System.out.println("DIGITE UM NÚMERO INTEIRO!");
                le.nextLine();
            }
        } while(true)
    }
}
```

Veja a imagem abaixo que ilustra a hierarquia de exceções em Java:

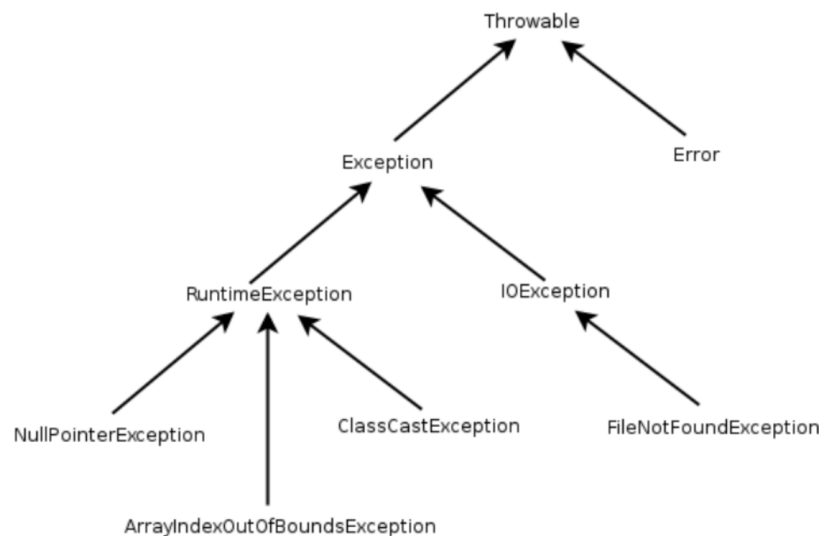


Figura 1.1: Hierarquia de exceções em Java

Fonte: Cabral (2014)

Além das maneiras de tratar exceções, é possível criar suas próprias exceções personalizadas, estendendo a classe **Exception** ou **RuntimeException**, dependendo se você

1 Organização do projeto, tratamento de exceções e conexão com banco de dados

deseja que seja uma checked ou unchecked exception. Isso permite que você defina regras específicas para o seu sistema e forneça mensagens de erro mais significativas para os usuários.

Para isso, você pode criar uma classe de exceção personalizada da seguinte forma:

```
public class MinhaExcecao extends RuntimeException {
    public MinhaExcecao(String mensagem) {
        super(mensagem);
    }
}
```

Um exemplo de uso dessa exceção personalizada seria uma verificação de saldo negativo em uma conta bancária:

```
public class ContaBancaria {
    private double saldo;
    public void sacar(double valor) {
        if (valor > saldo) {
            throw new MinhaExcecao("Saldo insuficiente para
                                   realizar o saque.");
        }
        saldo -= valor;
    }
}
```

O arquivo principal que utiliza a classe ContaBancaria poderia ser assim:

```
public class Principal {
    public static void main(String[] args) {

        ContaBancaria conta = new ContaBancaria();
        conta.depositar(50); // Deposito inicial de 50
        try {
            conta.sacar(100); // Tentativa de saque sem saldo
                             suficiente
        } catch (MinhaExcecao e) {
            System.out.println(e.getMessage());
        }
    }
}
```

No exemplo da clinica veterinária o uso de exceções ficaria assim:

```
public class Pet {
    private String nome;
    private int idade;

    public void setNome(String nome) {
        this.nome = nome;
    }
}
```

1 Organização do projeto, tratamento de exceções e conexão com banco de dados

```
// Método que valida a idade e pode lançar uma exceção
setIdade(int idade) {
    if (idade < 0) {
        // Lança uma exceção personalizada se a idade for inválida
        throw new IllegalArgumentException("A idade do pet não pode ser um valor negativo!");
    }
    this.idade = idade;
}

public void exibirInformacoes() {
    System.out.println("Pet: " + nome + " | Idade: " + idade + " anos");
}
}
```

Dessa forma se o usuário inserir um número negativo seria lançada uma exceção e impediria a continuação da execução do código.

```
public class Principal {
    public static void main(String[] args) {
        Pet meuPet = new Pet();
        meuPet.setNome("Thor");

        // Bloco try-catch para tratar possíveis erros de cadastro
        try {
            System.out.println("Tentando definir a idade do pet...");

            // Simulando um erro digitado pelo usuário (idade inválida)
            meuPet.setIdade(-3);

            // Esta linha não será executada se a exceção for lançada acima
            meuPet.exibirInformacoes();
        } catch (IllegalArgumentException e) {
            // O catch captura o erro e trata a falha com segurança
            System.out.println("Erro de validação" + e.getMessage());
            System.out.println("Por favor, corrija o valor e tente novamente.");
        } finally {
            // Bloco opcional que executa sempre (com ou sem erro)
            System.out.println("Fim da tentativa de cadastro do");
        }
    }
}
```

1 Organização do projeto, tratamento de exceções e conexão com banco de dados

```
        pet.");  
    }  
}  
}
```

Referências Bibliográficas

Cabral, L. P. (2014). Java e orientação a objetos.

Leite, T. et al. (2016). *Orientação a Objetos: Aprenda seus conceitos e suas aplicabilidades de forma efetiva*. Editora Casa do Código.